3-4

LangGraph 심화

학습 목차

- **Subgraph**
  - Subgraph
- **Persistence**
  - Checkpointer
  - Memory Store
- **Parallel Execution**
  - Fan-out, Fan-in
  - Send

© Scopelabs. All Rights Reserved

Subgraph

Subgraph

Subgraph: 독립적으로 컴파일한 그래프를 상위 그래프의 노드처럼 사용하는 구조

- 복잡한 워크플로우를 기능 단위로 나눌 수 있음
- Subgraph별 State를 따로 정의할 수 있음
- 요약, 번역, 문서 처리처럼 반복되는 흐름을 재사용하기 좋음

부모 그래프의 노드 내부에 자식 그래프를 호출하는 방식으로 만들어진다.

```
summary_subgraph = summary_builder.compile()

def summarize_document(state: ParentState) -> dict:
    result = summary_subgraph.invoke({
        "text": state["document"],
        "summary": "",
    })
    return {"summary": result["summary"]}

parent_builder.add_node("summarize_document", summarize_document)
```

Persistence

Checkpointer

Checkpointer: LangGraph 실행 중의 State를 저장하고 같은 thread에서 이어서 실행할 수 있게 하는 장치

- `MemorySaver`: 프로세스 메모리에 저장하는 개발/데모용 체크포인터
- `SqliteSaver`: SQLite 파일에 저장해 재시작 후에도 상태 유지
- `thread_id`로 사용자별, 세션별 실행 흐름을 구분

체크포인터를 이용하면 같은 `thread_id`의 이전 State를 이어받고, 실행 이력을 조회하거나 과거 체크포인트에서 다시 실행할 수 있다.

```
app = builder.compile(checkpointer=MemorySaver())
config = {"configurable": {"thread_id": "demo"}}

app.invoke({"messages": [HumanMessage(content="내 이름은 홍길동이야.")]}, config)
result = app.invoke({"messages": [HumanMessage(content="내 이름이 뭐야?")]}, config)
```

Memory Store

Memory Store: Checkpointer와 별도로 사용자별 장기 기억을 저장하는 공간

- thread를 넘어 공유되는 장기 정보를 저장
- `namespace`로 사용자, 조직, 프로젝트별 기억을 분리
- 사용자 선호, 반복 업무 규칙, 중요한 사실을 저장하는 데 적합

```
store = InMemoryStore()
app = builder.compile(checkpointer=MemorySaver(), store=store)

namespace = ("memories", user_id)
memories = store.search(namespace)

store.put(namespace, "fact_0", {
    "fact": "사용자의 취미는 피아노 연주",
})
```

Parallel Execution

Fan-out, Fan-in

Fan-out/Fan-in: 하나의 입력을 여러 작업으로 나누어 병렬 실행한 뒤 결과를 다시 합치는 패턴

- Fan-out: 독립적인 노드나 Worker로 작업을 분산
- Fan-in: 각 분기의 결과가 모이면 다음 노드에서 통합
- 장점/단점 분석, 여러 문서 요약, 다중 검색처럼 독립 작업에 적합
- 병렬 결과를 합치려면 State의 병합 규칙을 명확히 설계해야 함

```
builder.add_edge(START, "pros")
builder.add_edge(START, "cons")      # fan-out
builder.add_edge("pros", "summary")
builder.add_edge("cons", "summary")  # fan-in
builder.add_edge("summary", END)
```

Send

`Send`: 실행 시점에 필요한 Worker 수를 동적으로 결정하는 LangGraph의 병렬 실행 API

- `Send("worker", state)` 형태로 특정 노드에 별도 입력을 보냄
- 조건부 Edge에서 `list[Send]`를 반환하면 Worker가 병렬 생성
- 입력 리스트 길이에 따라 작업 수가 달라지는 상황에 유용
- 결과를 누적하려면 `Annotated` reducer 같은 병합 전략이 필요

```
def fan_out(state: DynamicState) -> list[Send]:
    return [
        Send("worker", {"query": q, "results": []})
        for q in state["queries"]
    ]

builder.add_node("worker", worker_node)
builder.add_conditional_edges(START, fan_out, ["worker"])
builder.add_edge("worker", END)
```